

TriGuard — Prototype Demo: Script & Commands

CM3070 Preliminary Report · Tier-A demonstration video · own voice · MP4

Hamza — BSc Computer Science, University of London

June 2026

How to use this sheet

Record the **Tier-A** prototype on the **Windows** **.venv** (no WSL, no environment flags). This matches the Preliminary Report. Read each **Voiceover** line in your own voice; run the **RUN** command on screen when shown. Target length **3–5 minutes** (script $\approx$ 3 min 23 s). Do not speed up or slow the audio. Export as **MP4 (H.264 + AAC)**.

Before you record (one-time setup)

Open PowerShell and prepare the session:

```
cd "C:\Users\javed\OneDrive\ICA EW SOFTWARE\FYP UOL\triguard"  
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass -Force  
.\.venv\Scripts\Activate.ps1  
$env:PYTHONPATH="src"
```

Your prompt should now show (.venv). If activation is blocked, skip it and prefix every command below with .\.venv\Scripts\python.exe instead of python.

Pre-flight — run once off-camera and confirm all green before recording:

```
python -m pytest -q  
python -m triguard.evaluation.run_eval  
python -m triguard.cli run data\sample_inputs\sample_harmful.json
```

Expect: 23 passed (plus a few skipped — those are the out-of-scope Tier-B slow tests, ignore them); Accuracy: 1.000; and risk_label: harmful, recommended_action: block.

Commands quick reference

Shot	RUN on screen
5 Tests	python -m pytest -q
6 Harmful	python -m triguard.cli run data\sample_inputs\sample_harmful.json
7 Failure	python -m triguard.cli run data\sample_inputs\sample_safe.json
8 Evaluation	python -m triguard.evaluation.run_eval

Shots 1–3 are slides; shot 4 shows the repo (tree /F src\triguard or your editor); shot 9 is the closing slide.

Shot-by-shot script

Shot 1 — Title card (8 s)

Screen: title slide — “*TriGuard — Preliminary Report Prototype Demo · Hamza · CM3070*”.

“Hi, I’m Hamza. This is the prototype-demonstration video for my CM3070 Preliminary Report. The project is **TriGuard** — a multimodal, explainable, locally deployable content-moderation pipeline.”

Shot 2 — Motivation (35 s)

Screen: slide with three bullets — *Harmful content increasingly crosses modalities · Existing tools are usually single-modality or closed-cloud · Regulators now expect rationales (DSA, UK Online Safety Act)*.

“The motivation is short. Harmful content increasingly crosses modalities — a meme combines image and text, a voice note layers over a violent visual. But the strongest moderation tools today are usually single-modality, like Google’s Perspective API, or closed cloud services that platforms cannot self-host. At the same time, the EU Digital Services Act and the UK Online Safety Act now expect moderation decisions to come with a statement of reasons that a human can audit. TriGuard is my attempt to address all three of those gaps at once, by orchestrating several pre-trained models with a local LLM judge that emits a written rationale.”

Shot 3 — Architecture recap (20 s)

Screen: show docs\architecture.png full-screen.

“The architecture is six layers. An input handler validates the payload; a dispatcher routes each modality to a wrapper; the wrappers return normalised Pydantic evidence; the judge reads the evidence and emits a structured decision with a rationale; and a side branch sends the same result into the evaluation harness.”

Shot 4 — Repo tour (25 s)

Screen: show the project tree.

RUN (optional):

```
tree /F src\triguard
```

“This is the Tier-A prototype I have built. Pydantic schemas are in `orchestrator/schemas.py`. The text wrapper in `models/text_model.py` is a real scikit-learn TF-IDF plus logistic-regression classifier trained on a small bundled corpus. The image and audio wrappers are mocks. The judge in `models/llm_judge.py` has two paths — a deterministic rule-based judge for this demo, and an Ollama path with retry-on-invalid-JSON and a graceful fallback to the rule judge. The orchestrator wires it together.”

Shot 5 — Run the tests (20 s)

RUN:

```
python -m pytest -q
```

“First the test suite. Twenty-three pytest cases run in about four seconds, covering schema bounds, the safe-cannot-block invariant, harmful-cannot-allow, partial-modality input, the orchestrator’s missing-input rejection and the Ollama-unavailable fallback path.”

Shot 6 — Run a harmful sample (35 s)

RUN, then highlight `risk_score`, `risk_label`, `recommended_action`, `rationale`:

```
python -m triguard.cli run data\sample_inputs\sample_harmful.json
```

“Now the pipeline. I’ll feed it a deliberately harmful payload — a threatening text, a photo described as containing a weapon, and an audio clip tagged as shouting. The system returns a risk score of 0.87, a risk label of *harmful*, all three modalities flagged, a recommended action of *block*, and a rationale that names the evidence: the text classifier’s score, the image caption and the audio event. Everything is schema-valid by construction because the Pydantic models reject inconsistent output.”

Shot 7 — A real failure case (25 s)

RUN, then pause on the *borderline* label + uncertainties:

```
python -m triguard.cli run data\sample_inputs\sample_safe.json
```

“It is more interesting to look at a failure case. On a perfectly benign sample the text classifier scores 0.37, the pipeline labels it *borderline* and recommends *review*. The label is wrong — the text is plainly safe — but notice that the system tags `low_text_classifier_confidence` in the uncertainties list and chooses *review* rather than *block*. That is the conservative behaviour the design asks for. The chapter explains that the root cause is a very small training corpus, and the fix is to swap in a Hugging Face toxicity model in the next iteration.”

Shot 8 — Evaluation results (20 s)

RUN, then show the printed metrics + the saved JSON path:

```
python -m triguard.evaluation.run_eval
```

“The evaluation harness drives fourteen hand-built tri-modal samples through the pipeline, reports per-class precision, recall and F1, plus p50 and p95 latency, and saves a full results envelope to disk. On this constructed set the orchestration combines evidence into the intended label every time. I am very careful to flag in the chapter that this is a *consistency check* on the engineering, not a generalisation claim.”

Shot 9 — Close (15 s)

Screen: back to the title slide.

“That is the Tier-A prototype: schemas, a real text classifier, mock image and audio wrappers, a working judge, twenty-three tests and a small reproducible evaluation. Replacing the mocks with BLIP, Whisper and Ollama is the work of the next phase. Thank you.”

Timing summary

Shot	Length
1 Title	8 s
2 Motivation	35 s
3 Architecture	20 s
4 Repo tour	25 s
5 Tests	20 s
6 Harmful sample	35 s
7 Failure case	25 s
8 Evaluation	20 s

Shot	Length
9 Close	15 s
Total	≈ 3 min 23 s

Inside the 3–5 minute window with room for natural pauses.

Recording checklist

- ☐ Title slide prepared.
- ☐ Terminal font ≥ 14 pt, dark background, window large enough to read.
- ☐ (`.venv`) active and `$env:PYTHONPATH="src"` set.
- ☐ Pre-flight green: 23 passed, Accuracy: 1.000, harmful $\rightarrow$ block.
- ☐ All three sample JSONs run end-to-end.
- ☐ Microphone tested; quiet room.
- ☐ Record **Tier-A only** — Windows `.venv`, no WSL, no `TRIGUARD_*_BACKEND` flags.
- ☐ Own voice; no AI voice; no speed change.
- ☐ Export **MP4 (H.264 + AAC)**, 1080p or 720p; verify it plays in VLC before upload.